Problem Statement 1: Smart Logistics Processing Pipeline

Scenario

LogisticsTech Systems runs an automated warehouse where incoming shipment requests are queued and processed sequentially using the **First-Come, First-Served (FCFS)** CPU scheduling algorithm. Each shipment job arrives at a specified time and requires a specific amount of CPU execution time (Burst Time) to allocate inventory, print shipping labels, and assign delivery drivers.

Your task is to write a C program to simulate the FCFS scheduling algorithm for N processes with varying **Arrival Times (AT)** and **Burst Times (BT)**.

Program Requirements

1. **Input:**
    - Number of processes (N).
    - Arrival Time (AT) and Burst Time (BT) for each process.
2. **Sorting/Ordering:**
    - Sort the processes based on their Arrival Time in ascending order. (If two processes arrive at the same time, process them in order of process ID).
3. **Handling CPU Idle Time:**
    - If the CPU is idle when a process arrives (i.e., process arrival time is greater than the current time), the system time must jump forward to match the arrival time of that process.

Problem Statement 2: Cloud Render Engine Job Scheduler

Scenario

An animation studio runs a dedicated 3D graphics rendering server called **RenderCore**. Animators submit render jobs throughout the day. To minimize overall wait times for the entire creative team, the server uses a **Non-Preemptive Shortest Job First (SJF)** CPU scheduling strategy.

When the rendering CPU becomes free, the job scheduler checks all jobs that have **already arrived** in the queue and selects the job with the **shortest Burst Time (BT)**. Once a rendering job starts, it executes to completion without interruption (non-preemptive). If a tie occurs in burst times among available jobs, the job that arrived first (FCFS tie-breaker) is selected.

Your task is to write a C program to simulate the Non-Preemptive SJF scheduling algorithm for N render tasks with varying **Arrival Times (AT)** and **Burst Times (BT)**.

Program Requirements

1. **Input:**
    - Total number of processes (N).
    - Arrival Time (AT) and Burst Time (BT) for each process.
2. **Scheduling Engine Logic:**
    - Keep track of the current time t.
    - At any time t, identify all processes that have arrived ($AT \leq t$) and are not yet completed.
    - Select the uncompleted process with the **minimum Burst Time**.
    - If no process has arrived at time t, increment t to the arrival time of the next earliest arriving process (handle CPU idle periods).
    - Execute the selected process to completion before re-evaluating the ready queue.